CRYPTOGRAPHY FUNDAMENTALS

Cryptography is the science of protecting information by converting it into a secure format that only authorized people can read.

Its main goal is to keep data **confidential, authentic and untampered**.

---

1. What is Cryptography?

Cryptography is the method of protecting data by converting it from normal, readable form (**plaintext**) into an unreadable form (**ciphertext**).

This prevents hackers and unauthorized people from understanding the data.

Example:

```
Hello → 8d9f2k#@9d
```

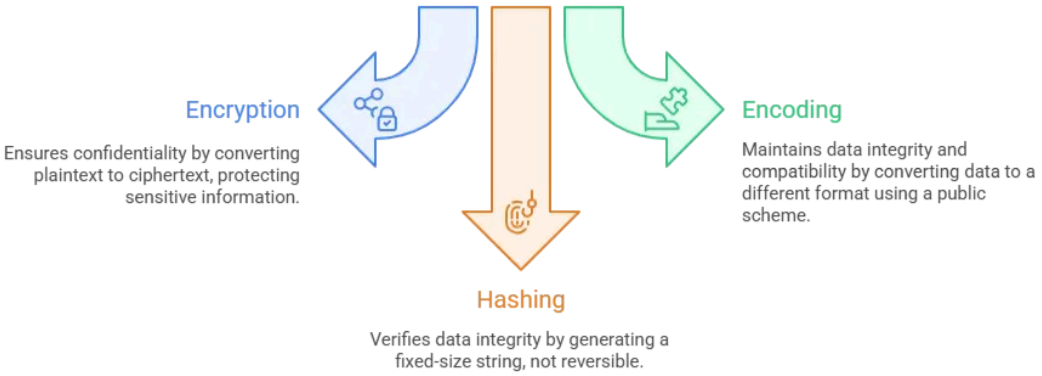
Only someone with the correct key can convert it back to:

```
8d9f2k#@9d → Hello
```

---

2. Encryption vs Encoding vs Hashing

Type	Purpose	Reversible	Example
Encryption	Security (Protection)	Yes (with key)	AES, RSA
Encoding	Data compatibility	Yes	Base64
Hashing	Data integrity	No	SHA-256



Easy Understanding

- **Encryption** = Locking data with a key

- **Encoding** = Changing data format for transport
- **Hashing** = Turning data into a digital fingerprint

---

3. Symmetric Encryption (AES)

Symmetric encryption uses **the same key** for both encryption and decryption.

Example algorithm:

AES (Advanced Encryption Standard)

How it works:

```
Plaintext + Secret key = Ciphertext
Ciphertext + Same key = Plaintext
```



Advantages:

- Very fast
- Very strong

Disadvantage:

- The key must be shared securely

Used in:

- Disk encryption (BitLocker, FileVault)
- VPN
- Database encryption

---

4. Asymmetric Encryption (RSA)

Asymmetric encryption uses **two different keys**:

Public key → Encrypt

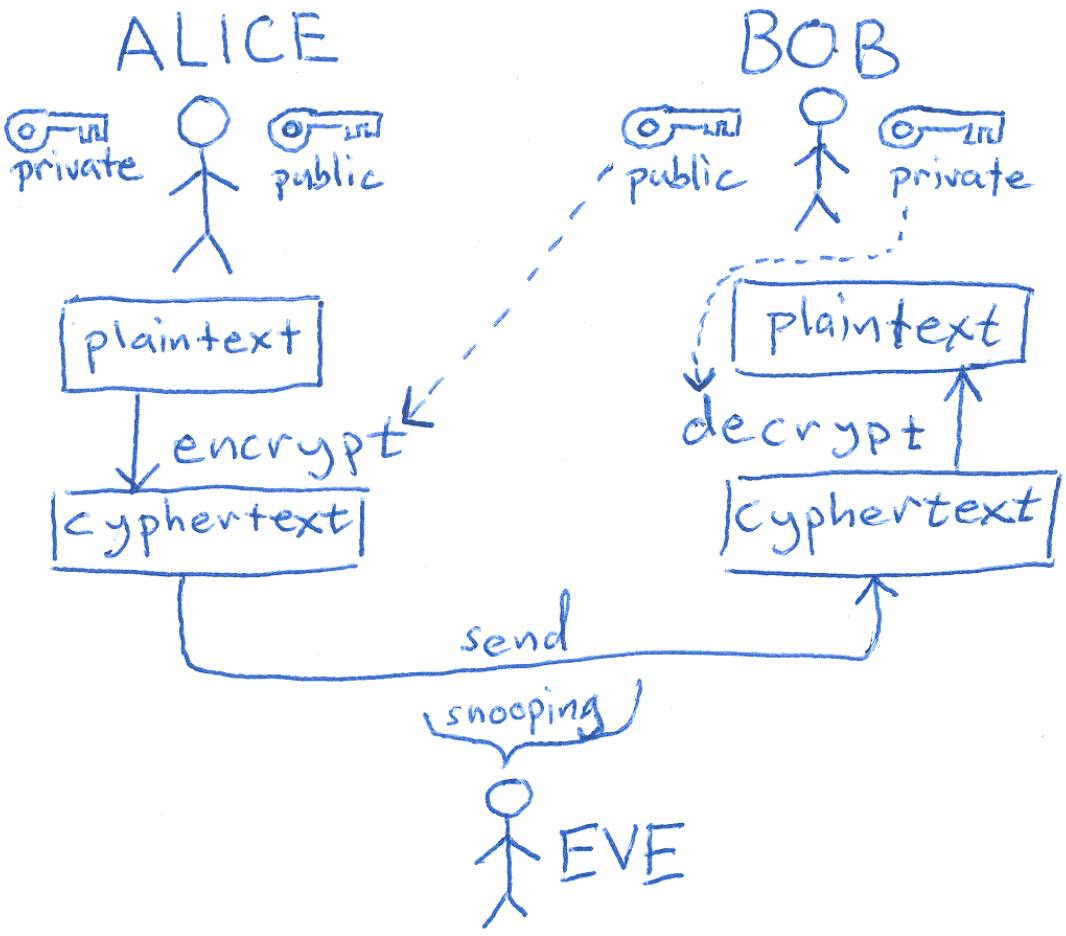
Private key → Decrypt

Example algorithm:

RSA

How it works:

> Sender → Encrypts using Public Key
> Receiver → Decrypts using Private Key



Advantages:

- No need to share private key

- Used for secure communication

Disadvantages:

- Slower than AES

Used in:

- HTTPS (websites)
- Digital certificates
- PGP/GPG email encryption
- SSH connections

---

5. Hash Functions

Hashing converts data into a **fixed-length hash value**.

It cannot be reversed.

Even a small change in the file creates a completely different hash.

Algorithm	Status
MD5	Broken / Unsafe
SHA-1	Weak
SHA-256	Secure and recommended

Example:

```
Hello → 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

This is called a **hash digest**.

---

6. Hash Collision

A collision happens when **two different files produce the same hash value**.

This is very dangerous because:

- A fake file can appear genuine
- Digital signatures can be bypassed

MD5 and SHA-1 are vulnerable to this.

That is why we use:

SHA-256 or SHA-3

---

7. Practical Lab (Linux / Kali / Windows WSL)

Step 1: Create a text file

```
echo "This is my secret data" > secret.txt
```

Step 2: Encrypt using GPG

```
gpg -c secret.txt
```

Output:

```
secret.txt.gpg
```

Write a password when asked.

Step 3: Decrypt the file

```
gpg secret.txt.gpg
```

Enter the same password to get original file.

Step 4: Generate a SHA-256 hash

```
sha256sum secret.txt
```

This creates a unique digital fingerprint for the file.

Step 5: Verify integrity

Run same command after transfer:

```
sha256sum secret.txt
```

Same hash → File is safe

Different hash → File has been changed